

Deterministic Environments & Containerized Jupyter

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India



What you will be able to do after this session

- Explain the three levels of pinning: ranges, exact versions, hashes.
- Generate a lockfile with pip-compile and consume it in a Dockerfile — hands-on.
- Pick compatible CUDA / driver / framework versions using the compatibility chain.
- Slim production images with multi-stage builds.
- Launch a GPU-backed Jupyter container with correct volumes and shared memory — hands-on.

Why 'pip install torch' is not reproducible

1 Unpinned

`pip install torch transformers`

Resolves to whatever is newest today. Transitive deps float too. Two builds a week apart can differ in dozens of packages.

2 Pinned + locked

`torch==2.11.2`, every transitive dep pinned, optionally with sha256 hashes. The same input always yields the same environment.

Reproducibility bug reports usually start with: 'nothing changed in our code'. Something changed in the resolver's world.

The pinning ladder

- 1 Ranges (`torch>=2.10,<3`) — flexible, drifts. OK for libraries, risky for training.
- 2 Exact pins (`torch==2.11.2`) — repeatable installs while wheels stay published.
- 3 Hash-locked (`--require-hashes`) — tamper-evident, supply-chain safe, CI-grade.

Lab 1 — Lockfile workflow with pip-compile

```
# requirements.in  (what you actually care about)
torch==2.11.*
transformers>=4.50
datasets

$ pip install pip-tools
$ pip-compile --generate-hashes requirements.in  # -> requirements.txt
$ head -4 requirements.txt
torch==2.11.2 \
    --hash=sha256:9f2c1a... \
    --hash=sha256:41d77b...

# In the Dockerfile:
RUN pip install --no-cache-dir --require-hashes -r requirements.txt
```

Intent lives in .in (human-edited); truth lives in .txt (generated, committed).

The compatibility chain

One line to remember

```
Host driver  >=  CUDA toolkit in image  ==  CUDA of framework wheel
      |                |                |
nvidia-smi   |  nvidia/cuda:13.0.0-*    |  torch==2.11.2+cu130
      v                v
      Python in image == Python the wheel was built for (3.12)
```

- Pin all four in writing: driver floor, CUDA tag, framework wheel, Python minor.
- Wheels encode their CUDA: the +cu130 suffix / index URL is part of the pin.
- Document the driver FLOOR in your README — it is the one thing the image cannot carry.

Multi-stage builds: compile fat, ship thin

```
# Stage 1 - builder: has nvcc, compiles custom kernels / wheels
FROM nvidia/cuda:13.0.0-devel-ubuntu24.04 AS builder
RUN pip wheel --wheel-dir /wheels -r requirements.txt

# Stage 2 - runtime: no compilers, just the artifacts
FROM nvidia/cuda:13.0.0-runtime-ubuntu24.04
COPY --from=builder /wheels /wheels
RUN pip install --no-index --find-links=/wheels -r requirements.txt
```

- Typical saving: devel (~7 GB) stays out of production; runtime image shrinks by gigabytes.
- Smaller image = faster node pull = faster autoscaling and faster job starts.

Containerized Jupyter: what actually needs wiring

Network

Publish the port:
-p 8888:8888
Bind to 0.0.0.0 inside;
use the token URL it
prints.

Data

Bind-mount work:
-v \$PWD:/workspace
Notebooks survive the
container; images stay
stateless.

Memory

--shm-size=8g
DataLoader workers
exchange tensors via
/dev/shm; the 64 MB
default crashes them.

*The container is disposable. The
volume is not. Treat every
notebook server as cattle, not a
pet.*

Lab 2 — Launch a GPU-backed Jupyter

```
docker run --rm -it \  
  --gpus all \  
  --shm-size=8g \  
  -p 8888:8888 \  
  -v "$PWD":/workspace -w /workspace \  
  quay.io/jupyter/pytorch-notebook:cuda13-latest \  
  start-notebook.py --IdentityProvider.token='lab2'
```

```
# Open http://localhost:8888/?token=lab2 then in a cell:  
import torch; torch.cuda.get_device_name(0)
```

- Verify: GPU name prints; a file saved in /workspace appears on the host.
- Stop and relaunch — the notebook survives. That is the volume at work.

Where should the code live?

Bind-mounted (dev)

Edit on host, run in container, instant feedback. Pair with an editor that attaches to containers (e.g. devcontainers).

Baked in (ship)

COPY the code at build time. The image alone is the deployable, auditable artifact — nothing depends on a host checkout.

**Same image, two modes:
mount over /workspace
during the day, rebuild to
freeze a version.**

Hygiene: .dockerignore and secrets

.dockerignore

```
# .dockerignore
.git
data/          # datasets do not belong in images
checkpoints/
*.ipynb_checkpoints
.env           # secrets NEVER enter the build context
```

- Everything in the build context is sent to the daemon — a stray 200 GB data/ makes builds crawl.
- Secrets in ENV or COPY persist in layers forever, even if 'deleted' later.
- Use build secrets instead: RUN --mount=type=secret ,id=hf_token ... (never written to a layer).

Exercise & check yourself

Exercise

Convert Session 1's image to a locked build: `requirements.in` → `pip-compile --generate-hashes` → `--require-hashes install`. Then launch Jupyter from it with GPU, volume, and 8 GB shm.

Check-for-understanding

1. What can change between two unpinned builds run a month apart?
2. Why must the wheel's CUDA suffix match the image's toolkit?
3. Why does the DataLoader care about `--shm-size`?

Recap & what's next

- Intent in .in, truth in a hash-locked .txt — generated, committed, reviewed.
- Pin the whole chain: driver floor, CUDA tag, framework wheel, Python minor.
- Multi-stage: compile in devel, ship on runtime.
- Jupyter needs three wires: port, volume, shared memory.
- **Next — Session 3: a CUDA primer inside containers, and CI that validates every image build.**